CSC3067: Group 17 Project Report

Aaron McGee	40367926	amcgee05@qub.ac.uk
John Scott	40370740	jscott87@qub.ac.uk
Michal Wisniewski	40373624	mwisniewski02@qub.ac.uk

Table of Contents:

Introduction

The main goal of this work is to construct a model capable of reliably identifying pedestrians under varying conditions, such as noisy images or changes in an image's lighting, while also reducing the computational cost and increasing the accuracy of the model.

The first section of this report focuses on the training of the model, covering the data preprocessing techniques used, the model's classifier, training methodology, and parameters used to develop the model. This is followed by a section on testing, in which we assess the model's performance using varying metrics and parameters to assess its robustness under certain conditions. Next, we discuss the detection implementation for our trained model, using techniques such as sliding window and non-maxima suppression. Finally, we conclude the report by discussing what we felt went well when training and testing the model, how it could be improved and what other techniques we could have used to optimize the model.

Training

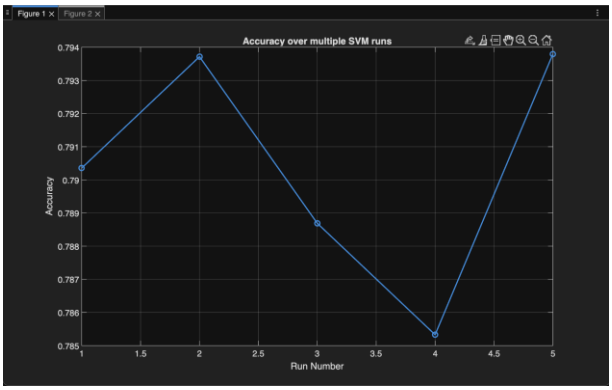
Feature Descriptor

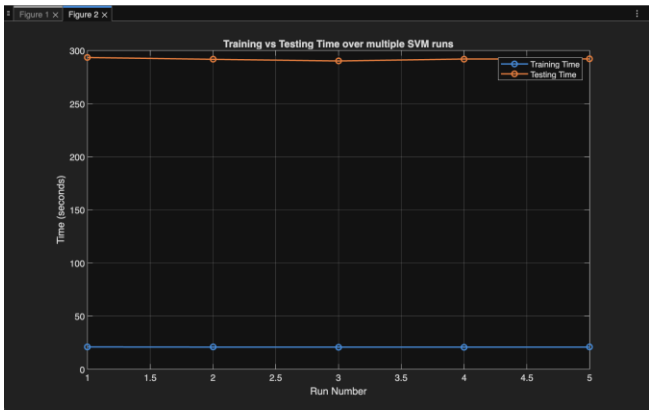
Full Image

Full image in our testing seemed to be the worst option for feature extraction, due to a lower accuracy when using raw pixel extraction, and a significantly increased training time in comparison to other feature extraction techniques.

As we can see below, while testing full image extraction on our SVM model, the mean accuracy for this technique comes to 79.1%. The mean training time comes to 19 seconds. The mean testing time comes to 291 seconds.

SVM Model with full image feature extraction:





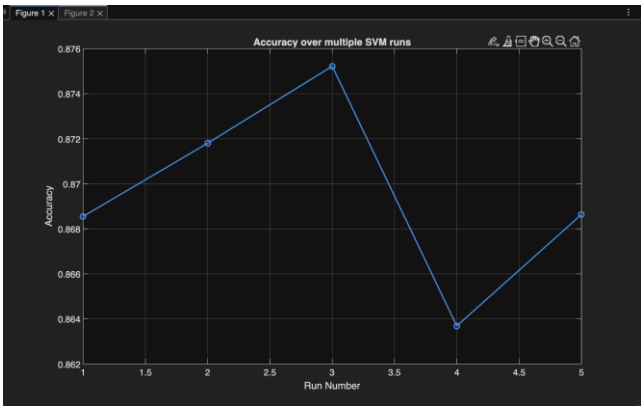
Histogram of Gradients (HOG)

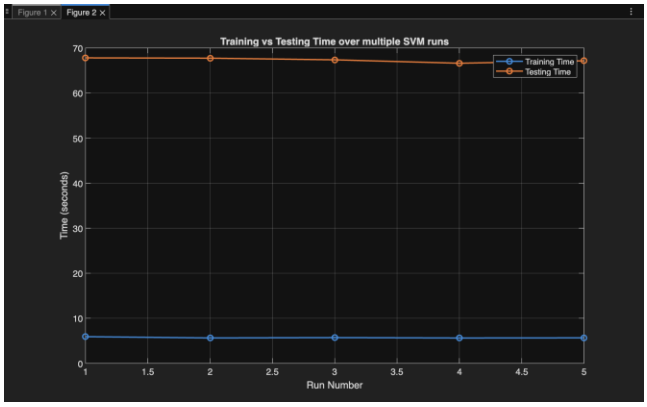
We used HOG in our final pedestrian detection model, due to the increase in training speed alongside an increase in the model's accuracy. By allowing HOG to look at a summary of the gradients in the image, it allows the model to more easily and more quickly outline the structure of an object in the image and identify if it is like the training images provided to the model.

We also chose HOG as it works well with SVM given that HOG produces a linearly separable output which SVM can separate using a hyperplane. HOG was also designed with pedestrian detection in mind, which suits this project.

As we can see below, while HOG is applied to our SVM model, the mean accuracy for this technique comes to 87.0%. The mean training time comes to 5.7 seconds. The mean testing time comes to 67.3 seconds.

VM Model with HOG feature extraction:



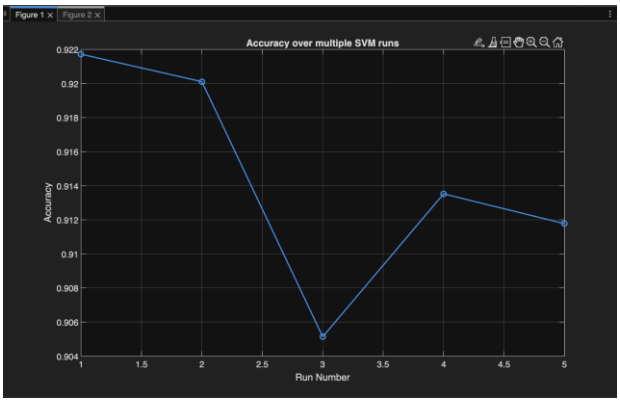


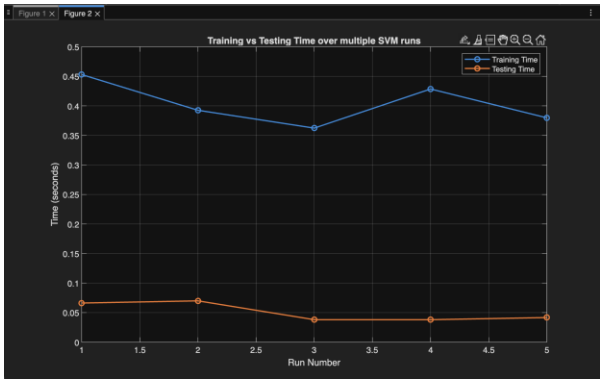
Dimensionality Reduction Techniques

We implemented some forms of dimensionality reduction techniques. For example, we used PCA to reduce the training and testing times of our model. It allows us to run our model at a higher sampling rate for less computational cost.

As we can see below, while testing PCA on our SVM model with full image extraction, the mean accuracy for this technique comes to 91.4%. The mean training time comes to 0.4 seconds. The mean testing time comes to 0.051 seconds.

SVM Model with PCA (HOG feature extraction):





Learning Method

Nearest Neighbour and kNN

We used the following setup parameters as we believe they paint the most accurate and fair picture: iterations: 50, sampling: 5, k: 10

NN:

Label	trainingTime	testingTime	TP	TN	FP	FN	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Mean Values:	7.0336e-05	4.6184	88.92	89.62	10.74	111.72	0.59316	0.40684	0.89328	0.89295	0.44354	0.58941	0.10705

kNN:

Label	trainingTime	testingTime	TP	TN	FP	FN	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Mean Values:	7.7402e-05	4.9315	130.14	84	16.74	70.12	0.71143	0.28857	0.88888	0.83579	0.65096	0.74685	0.16421

Having a look at the above NN/kNN results, it can be observed that the results match our findings for the online presentation; both classifiers have a high a very low training time and the testing time increases exponentially the lower the sampling rate, resulting in a high combined training/testing time. NN has rather poor confusion matrix results, while kNN improves them while retaining a similar combined training/testing time. From the data, the accuracy increases from 59.3% to 71.1% when using the K- nearest neighbors and all other metrics improve except for the specificity which decreases from 89.3% to 83.5%. This is there due to there being more false positives as K-NN uses an average of multiple neighbors making it more likely to misclassify negative samples as positive.

SVM

We decided to use SVM as we believe it is the best suited learning method for this project. SVM, when paired with HOG, should theoretically be better than other learning methods given that they complement each other perfectly and are a standard for pedestrian detection, having been used in the industry for two decades.

We implemented SVM (and NN/kNN in a similiar manner) with the following file structure: SVMTraining, SVMTesting, SVMClassifierFunction, SVMClassifierScript. SVMTraining and SVMTesting are self-explanatory; they train and test the SVM classifier and are both ran by the SVMClassifierFunction, which does the main legwork by actually running the classifier. SVMClassifierScript on the other hand is a script that allows the user to tailor the parameters used by the SVM classifier and outputs the results in a human-readable format in the form of a detailed csv file/table and some graphs.

We changed the following SVM parameters: sampling rate, kernel, lambda, C, sigmakernel, k, ndim, threshold, and recorded the results of each change. These results are saved in the table below.

O	P	Q	R	S	T	U	V	W
iterations	**sampling**	**kernel**	**lambda**	**C**	**sigmakernel**	**k**	**ndim**	**threshold**
Number	Number	Text	Number	Number	Number	Number	Number	Number
iterations	sampling	kernel	lambda	C	sigmakernel	k	ndim	threshold
1	**50**	**gaussian**	**1e-20**	**120**	**10**	**4**	**70**	**0.9**

Overview of each parameter

Sampling Rate: The rate at which the classifier loads the images, e.g 10 would be 1 out of every 10 images is loaded
Kernel: The kernel used for the SVM parameter
Lambda: Regularization parameter controlling the trade-off between margin size and training error.
C: SVM penalty parameter controlling misclassification tolerance.
Sigmakernel: Kernel-specific parameter controlling width in RBF kernels.
K: Number of neighbors used to average instead of just using Nearest Neighbor.
Ndim: Number of dimensions that PCA reduces to.
Threshold: Decision boundary threshold for classification.

Pre-Processing

For our SVM pre-processing we decided to go with a few standard pre-processing methods to improve our SVM classifier results:

```
% Preprocessing
% Z-score standardization
% Prevents features with large numeric scales from dominating
mu = mean(trainImages, 1);
sigma = std(trainImages, 0, 1);
sigma(sigma == 0) = 1;

trainImages = (trainImages - mu) ./ sigma;
testImages = (testImages - mu) ./ sigma;

% L2-normalize each HOG vector
% Makes HOG descriptors comparable across images
trainImages = trainImages ./ vecnorm(trainImages, 2, 2);
testImages  = testImages  ./ vecnorm(testImages, 2, 2);

% Replace NaNs (if any rows become 0 vector after normalization)
% Shouldn't happen, but best to stay on the safe side
trainImages(any(isnan(trainImages),2), :) = 0;
testImages(any(isnan(testImages),2), :) = 0;

% PCA
% Compute PCA from training data only
[PCAVectors, ~, PCAMean, Xtrain_pca] = pca(trainImages, ndim);

% Project testImages into PCA space
Xtest_pca = (testImages - PCAMean) * PCAVectors;
```

Below are some results taken of the SVM classifier with the pre-processing turned off:

Label	trainingTime	testingTime	TP	TN	FP	FN	accuracy	errorRate	precision	specificity	sensitivity	fMeasure
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Mean Values:	0.87038	40.7574	68.05	50	0	32.2	0.78566	0.21434	1	1	0.67874	0.80768

Here is a run with pre-processing turned on:

Label	trainingTime	testingTime	TP	TN	FP	FN	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Mean Values:	0.073645	0.028243	71.208	39.812	0.188	8.992	0.92363	0.076368	0.99743	0.9953	0.88789	0.93909	0.0047

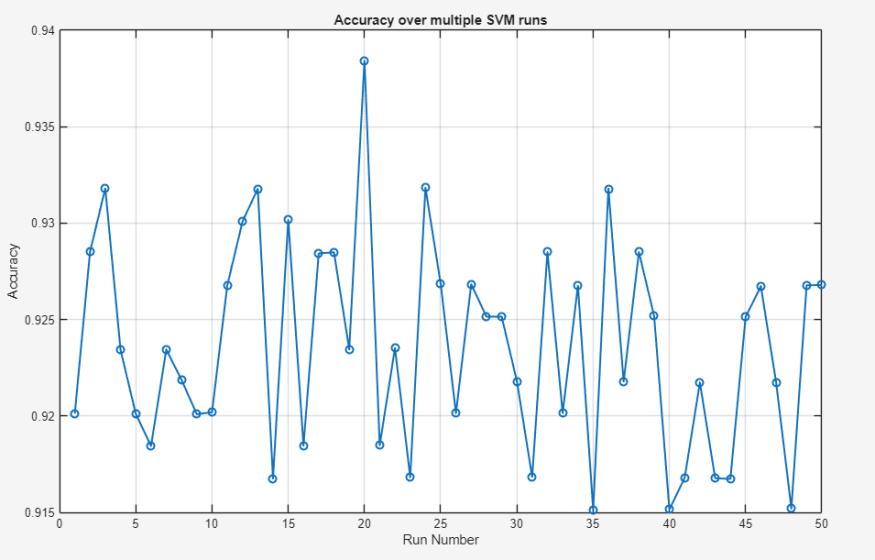
As can be seen by the above results, SVM with pre-processing is superior. This is because with no Z-score standardization features with larger numeric ranges dominate the SVM decision boundary, ignoring potentially important features with smaller scales. No L2 normalization also means that illumination variations cause the same object to have different feature magnitudes, confusing the classifier and no PCA means that the classifier is working in full dimensionality, introducing noise and making optimization harder. This results in a generally worse classifier output, with confusion matrix values seeing a ~14% decrease across the board. Testing time also sees a large spike, which is likely a cause of no PCA being in place.

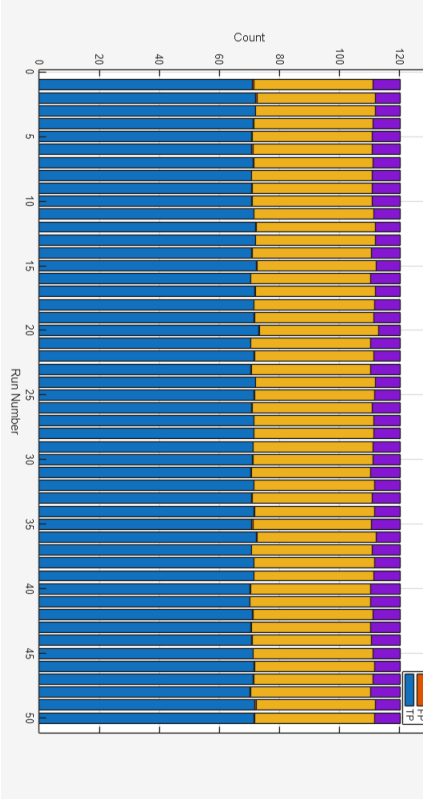
(On a sidenote: we realized that we handled the training time and testing time incorrectly for SVM; we took the mean train/test time from the cross-validation, resulting in train/test value that is a 1/5th of what it should be. To see the real value, you should multiply the train/test time by 5 to offset the mean.)

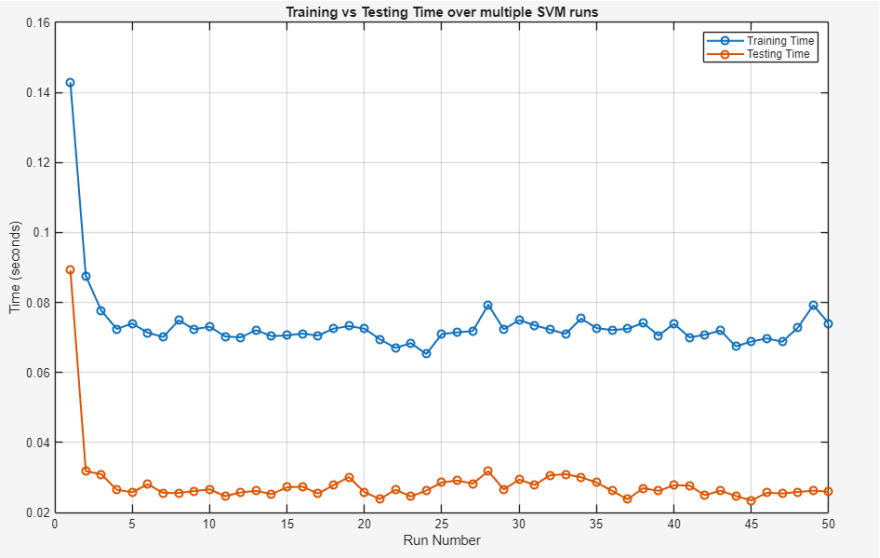
Running SVM

When running the model 50 times, and with a sampling rate of 5 to match the KNN and NN runs, we get the following output.

Label	trainingTime	testingTime	TP	TN	FP	FN	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Mean Values:	0.073645	0.028243	71.208	39.812	0.188	8.992	0.92363	0.076368	0.99743	0.9953	0.88789	0.93909	0.0047







Training evaluation

As can be observed in the above tables, while NN and kNN have a very short training time, there is a rather high testing time. SVM in comparison has a much lower testing time and

while having a higher training time than NN/kNN, the total training/testing time is lower than NN/kNN.

Our conclusion is also supported by the confusion matrix values where it can be observed that SVM outperforms NN/kNN, where SVM beats the accuracy of kNN by 21% and the fmeasure by 19%. This is no surprise because, as discussed previously, SVM with HOG are designed with pedestrian detection in mind.

Testing

Split

For our split, we decided to use cross-validation rather than 50/50. This is because, while cross-validation is more computationally expensive given that it requires training and evaluating the model multiple times across different folds, it provides a far more reliable performance estimate than a single 50/50 split. The only circumstance where 50/50 is better suited than cross-validation is when a huge dataset is used. For our project, we have 3632 negative and 3633 positive images, which is too low as 50/50 would only train on 3632 images, meaning the result would be highly dependent on how lucky we got in that 50/50 split. Our decision to use cross-validation rather than a 50/50 split is reinforced by the below findings:

50/50:

Label	trainingTime	testingTime	TP	TN	FP	FN	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Mean Values:	0.0049615	0.003748	34.408	20.124	0.124	5.544	0.18117	0.018831	0.19928	0.1988	0.17228	0.18476	0.0012014

Cross-Validation:

Label	trainingTime	testingTime	TP	TN	FP	FN	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Mean Values:	0.073645	0.028243	71.208	39.812	0.188	8.992	0.92363	0.076368	0.99743	0.9953	0.88789	0.93909	0.0047

As can be seen, while 50/50 has a slightly faster combined training/testing time than cross-validation (0.0087095 vs 0.0101888), the difference is minimal and therefore unimportant. The main difference can be seen in the confusion matrix values where 50/50 has an unusable result.

Performance Evaluation

As part of our performance evaluation, we decided to create a script which runs the classifier an *x* number of times (iterations) with whatever parameters the user supplies. Then, the results are saved into a table/csv for ease of viewing. The resulting csv contains data from each run, as well as a calculated average (mean) of all runs.

We decided to include training and testing time, as well as the resulting confusion matrix data: TP, FP, TN, FN, as well as: accuracy, error rate, precision, specificity, f measure and false alarm rate.
Lastly, we included the pre-requisite parameters for running the classifier. For SVM, these are the aforementioned iterations, sampling rate, kernel, lambda, C, sigma kernel, k, ndim and threshold.

The higher the iterations, the better; a higher number of iterations allows us to get more accurate, stable mean values as it allows us to minimize randomness. We decided on an iteration value of 5 because we found it resulted in a good average while keeping computational cost in check. A sampling rate of 5 is ideal for the same reasons why we decided on an iterations value of 5; it balances computational cost with a stable, reliable result.

The initial values we used for the beginning of our evaluation are as follows:

```
kernel = gaussian;
lambda = 1e-20;
C = 60;
sigmakernel = 20;
k = 5;
ndim = 70;
threshold = .5;
```

We then focused on each parameter one at a time to find the optimum value for each.

Kernel

Initial kernel of “poly” used:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.45359	0.011886	0.47934	0.52066	1	1	0.21975	0.35362	0
Run 2	0.55194	0.0070513	0.48899	0.51101	1	1	0.23398	0.3642	0
Run 3	0.47991	0.0040677	0.50252	0.49748	1	1	0.25441	0.37315	0
Run 4	0.42767	0.0042956	0.46744	0.53256	1	1	0.20173	0.33019	0
Run 5	0.53944	0.0051578	0.44087	0.55913	1	1	0.16194	0.27545	0
Mean Values:	0.49051	0.0064917	0.47583	0.52417	1	1	0.21436	0.33932	0

Then “gaussian”:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.053619	0.015054	0.9118	0.088196	0.98928	0.98	0.87778	0.92915	0.02
Run 2	0.033395	0.0065351	0.91351	0.086488	0.9861	0.975	0.88284	0.93139	0.025
Run 3	0.034924	0.0045478	0.90686	0.09314	0.98393	0.97	0.87543	0.92607	0.03
Run 4	0.033616	0.0047041	0.90844	0.091556	0.98142	0.965	0.88019	0.92747	0.035
Run 5	0.03343	0.0047377	0.91351	0.086488	0.98388	0.97	0.88534	0.93145	0.03
Mean Values:	0.037797	0.0071158	0.91083	0.089174	0.98492	0.972	0.88031	0.9291	0.028

Then “polyhomog”:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.19104	0.014054	0.66722	0.33278	0.66722	0	1	0.8004	1
Run 2	0.12996	0.0073885	0.66722	0.33278	0.66722	0	1	0.8004	1
Run 3	0.14801	0.0045331	0.66722	0.33278	0.66722	0	1	0.8004	1
Run 4	0.13256	0.005823	0.66722	0.33278	0.66722	0	1	0.8004	1
Run 5	0.14893	0.0051007	0.66722	0.33278	0.66722	0	1	0.8004	1
Mean Values:	0.1501	0.00738	0.66722	0.33278	0.66722	0	1	0.8004	1

We found that, even though it is far more computationally expensive than the other two, that the gaussian is the most optimal kernel to use.

Lambda

Initial value of lambda =1e-20:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.053619	0.015054	0.9118	0.088196	0.98928	0.98	0.87778	0.92915	0.02
Run 2	0.033395	0.0065351	0.91351	0.086488	0.9861	0.975	0.88284	0.93139	0.025
Run 3	0.034924	0.0045478	0.90686	0.09314	0.98393	0.97	0.87543	0.92607	0.03
Run 4	0.033616	0.0047041	0.90844	0.091556	0.98142	0.965	0.88019	0.92747	0.035
Run 5	0.03343	0.0047377	0.91351	0.086488	0.98388	0.97	0.88534	0.93145	0.03
Mean Values:	0.037797	0.0071158	0.91083	0.089174	0.98492	0.972	0.88031	0.9291	0.028

If we use a much bigger lamda = 1e-15:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.055572	0.018938	0.90685	0.093154	0.98341	0.97	0.87534	0.92615	0.03
Run 2	0.032543	0.0092985	0.91012	0.089876	0.98886	0.98	0.87528	0.92831	0.02
Run 3	0.036051	0.0043609	0.91017	0.089835	0.98102	0.965	0.88281	0.92888	0.035
Run 4	0.035658	0.0050274	0.9035	0.096501	0.98643	0.975	0.86781	0.92259	0.025
Run 5	0.044075	0.0054574	0.89847	0.10153	0.98666	0.975	0.86031	0.9182	0.025
Mean Values:	0.04078	0.0086165	0.90582	0.094179	0.98528	0.973	0.87231	0.92483	0.027

If we use a much smaller lamda =1e-25:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.053502	0.014588	0.90519	0.094807	0.97819	0.96	0.87793	0.92497	0.04
Run 2	0.034056	0.0076812	0.90357	0.096433	0.98818	0.98	0.86549	0.92223	0.02
Run 3	0.033682	0.0044191	0.91176	0.088237	0.98676	0.975	0.88015	0.92986	0.025
Run 4	0.043242	0.0045589	0.91014	0.089862	0.9836	0.97	0.88031	0.92861	0.03
Run 5	0.041948	0.0051752	0.90342	0.096584	0.98557	0.975	0.86769	0.92191	0.025
Mean Values:	0.041286	0.0072844	0.90682	0.093185	0.98446	0.972	0.87431	0.92552	0.028

We found that the optimal value was 1e-20.

C

Initial value of C = 60:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.053619	0.015054	0.9118	0.088196	0.98928	0.98	0.87778	0.92915	0.02
Run 2	0.033395	0.0065351	0.91351	0.086488	0.9861	0.975	0.88284	0.93139	0.025
Run 3	0.034924	0.0045478	0.90686	0.09314	0.98393	0.97	0.87543	0.92607	0.03
Run 4	0.033616	0.0047041	0.90844	0.091556	0.98142	0.965	0.88019	0.92747	0.035
Run 5	0.03343	0.0047377	0.91351	0.086488	0.98388	0.97	0.88534	0.93145	0.03
Mean Values:	0.037797	0.0071158	0.91083	0.089174	0.98492	0.972	0.88031	0.9291	0.028

If we use a bigger value of C =120:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.056299	0.012587	0.90847	0.091529	0.99139	0.985	0.87031	0.92685	0.015
Run 2	0.028122	0.004435	0.90183	0.098168	0.99163	0.985	0.86037	0.92099	0.015
Run 3	0.032516	0.0031916	0.90511	0.09489	0.99154	0.985	0.86522	0.92382	0.015
Run 4	0.030647	0.0029458	0.90847	0.091529	0.9917	0.985	0.87031	0.92665	0.015
Run 5	0.027425	0.0029013	0.90012	0.099876	0.98864	0.98	0.86028	0.91978	0.02
Mean Values:	0.035002	0.0052122	0.9048	0.095198	0.99098	0.984	0.8653	0.92362	0.016

If we use a smaller value of C = 5:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.056299	0.012587	0.90847	0.091529	0.99139	0.985	0.87031	0.92685	0.015
Run 2	0.028122	0.004435	0.90183	0.098168	0.99163	0.985	0.86037	0.92099	0.015
Run 3	0.032516	0.0031916	0.90511	0.09489	0.99154	0.985	0.86522	0.92382	0.015
Run 4	0.030647	0.0029458	0.90847	0.091529	0.9917	0.985	0.87031	0.92665	0.015
Run 5	0.027425	0.0029013	0.90012	0.099876	0.98864	0.98	0.86028	0.91978	0.02
Mean Values:	0.035002	0.0052122	0.9048	0.095198	0.99098	0.984	0.8653	0.92362	0.016

At this stage we realized the issue with the incorrect train/test times for SVM and fixed it in the codebase. From now on, the correct train/test values will be displayed.

We found that the optimal value was 40:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.27595	0.081141	0.91842	0.081584	0.97307	0.95	0.90262	0.93619	0.05
Run 2	0.18797	0.041901	0.92021	0.079793	0.97313	0.95	0.90537	0.93786	0.05
Run 3	0.16555	0.022267	0.9135	0.086501	0.9708	0.945	0.89784	0.93226	0.055
Run 4	0.16322	0.022177	0.91351	0.086488	0.97599	0.955	0.89287	0.93165	0.045
Run 5	0.17851	0.022326	0.91019	0.089807	0.97837	0.96	0.88537	0.92892	0.04
Mean Values:	0.19424	0.037962	0.91517	0.084835	0.97428	0.952	0.89681	0.93338	0.048

Sigmakernel

Initial value of Sigmakernel = 20:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.2824	0.07921	0.91515	0.084848	0.97056	0.945	0.90022	0.93376	0.055
Run 2	0.16985	0.044338	0.90685	0.093154	0.96558	0.935	0.89281	0.92749	0.065
Run 3	0.20362	0.025094	0.91176	0.088237	0.96794	0.94	0.89769	0.93106	0.06
Run 4	0.18133	0.025973	0.91186	0.08814	0.97324	0.95	0.89284	0.93059	0.05
Run 5	0.19065	0.030766	0.91522	0.08478	0.96625	0.935	0.9054	0.93362	0.065
Mean Values:	0.20557	0.041076	0.91217	0.087832	0.96871	0.941	0.89779	0.9313	0.059

Smaller value of Sigmakernel = 5:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.33575	0.070021	0.92517	0.074835	0.99455	0.99	0.89287	0.94069	0.01
Run 2	0.24966	0.033098	0.91848	0.081515	0.98893	0.98	0.88784	0.93557	0.02
Run 3	0.20386	0.02108	0.9318	0.068196	0.9944	0.99	0.90278	0.94621	0.01
Run 4	0.19221	0.01996	0.9335	0.066501	0.99456	0.99	0.90537	0.9476	0.01
Run 5	0.1847	0.019751	0.93675	0.063251	1	1	0.90519	0.94991	0
Mean Values:	0.23324	0.032782	0.92914	0.07086	0.99449	0.99	0.89881	0.94399	0.01

Bigger value of Sigmakernel = 40:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.44618	0.15499	0.72044	0.27956	0.70483	0.16	1	0.82683	0.84
Run 2	0.18355	0.044923	0.7204	0.2796	0.70506	0.16	1	0.82691	0.84
Run 3	0.1851	0.022354	0.72212	0.27788	0.70608	0.165	1	0.82769	0.835
Run 4	0.15924	0.021444	0.72377	0.27623	0.70751	0.17	1	0.82862	0.83
Run 5	0.17074	0.022921	0.72048	0.27952	0.70478	0.16	1	0.82682	0.84
Mean Values:	0.22896	0.053327	0.72144	0.27856	0.70565	0.163	1	0.82737	0.837

We found that the optimal value was 5.

K

Initial value of K = 5:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.26764	0.055005	0.92682	0.073182	0.98929	0.98	0.90031	0.94242	0.02
Run 2	0.12714	0.0165	0.93172	0.068278	0.9973	0.995	0.90012	0.94556	0.005
Run 3	0.15039	0.011173	0.92846	0.071543	0.99444	0.99	0.89778	0.94358	0.01
Run 4	0.12627	0.011079	0.9235	0.076501	0.99452	0.99	0.89034	0.93936	0.01
Run 5	0.12292	0.010979	0.93011	0.06989	0.99733	0.995	0.89775	0.94469	0.005
Mean Values:	0.15887	0.020947	0.92812	0.071879	0.99458	0.99	0.89726	0.94312	0.01

Bigger value of K = 10:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.4309	0.049577	0.94342	0.056585	0.99744	0.995	0.91768	0.95559	0.005
Run 2	0.34758	0.019319	0.93178	0.068224	1	1	0.89774	0.94507	0
Run 3	0.30945	0.015391	0.93683	0.063169	0.9973	0.995	0.9078	0.95009	0.005
Run 4	0.2913	0.015319	0.93503	0.064973	1	1	0.90256	0.94819	0
Run 5	0.3415	0.015813	0.93016	0.069836	0.99444	0.99	0.9003	0.94464	0.01
Mean Values:	0.34415	0.023084	0.93544	0.064557	0.99783	0.996	0.90522	0.94872	0.004

Smaller value of K = 2:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.11035	0.055769	0.91512	0.084878	0.99435	0.99	0.87776	0.93233	0.01
Run 2	0.078267	0.013986	0.92013	0.079873	0.98642	0.975	0.89274	0.93713	0.025
Run 3	0.071205	0.0098008	0.90683	0.093173	0.99148	0.985	0.86785	0.92552	0.015
Run 4	0.043513	0.0093479	0.92346	0.076539	0.99721	0.995	0.88779	0.93931	0.005
Run 5	0.05225	0.013087	0.91683	0.083167	0.99457	0.99	0.88037	0.93364	0.01
Mean Values:	0.071118	0.020398	0.91647	0.083526	0.9928	0.987	0.8813	0.93359	0.013

We found that the optimal value was 10.

NDim

Initial value of NDim = 70:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.4309	0.049577	0.94342	0.056585	0.99744	0.995	0.91768	0.95559	0.005
Run 2	0.34758	0.019319	0.93178	0.068224	1	1	0.89774	0.94507	0
Run 3	0.30945	0.015391	0.93683	0.063169	0.9973	0.995	0.9078	0.95009	0.005
Run 4	0.2913	0.015319	0.93503	0.064973	1	1	0.90256	0.94819	0
Run 5	0.3415	0.015813	0.93016	0.069836	0.99444	0.99	0.9003	0.94464	0.01
Mean Values:	0.34415	0.023084	0.93544	0.064557	0.99783	0.996	0.90522	0.94872	0.004

Bigger value of NDim = 150:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.47142	0.071021	0.93669	0.063306	1	1	0.90506	0.94914	0
Run 2	0.32325	0.06765	0.93008	0.069918	1	1	0.89518	0.94385	0
Run 3	0.38207	0.037719	0.92678	0.073224	0.99706	0.995	0.89274	0.94107	0.005
Run 4	0.37695	0.039915	0.94011	0.059891	1	1	0.91024	0.95247	0
Run 5	0.38998	0.040393	0.93014	0.069863	0.99737	0.995	0.8978	0.94458	0.005
Mean Values:	0.38873	0.05134	0.93276	0.06724	0.99889	0.998	0.90021	0.94622	0.002

Smaller value of NDim = 20:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.39192	0.040349	0.93008	0.069918	0.99473	0.99	0.90018	0.9446	0.01
Run 2	0.23779	0.008959	0.92678	0.073224	0.99174	0.985	0.89774	0.94216	0.015
Run 3	0.26005	0.0069041	0.93344	0.066557	0.99437	0.99	0.90524	0.94728	0.01
Run 4	0.22925	0.0061842	0.93339	0.066612	0.99737	0.995	0.90262	0.94716	0.005
Run 5	0.26765	0.0063081	0.93186	0.068142	0.99737	0.995	0.90043	0.94588	0.005
Mean Values:	0.27733	0.013741	0.93111	0.068891	0.99512	0.991	0.90124	0.94542	0.009

We found that the optimal value was 70.

Threshold

Initial value of Threshold = .6:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.47142	0.071021	0.93669	0.063306	1	1	0.90506	0.94914	0
Run 2	0.32325	0.06765	0.93008	0.069918	1	1	0.89518	0.94385	0
Run 3	0.38207	0.037719	0.92678	0.073224	0.99706	0.995	0.89274	0.94107	0.005
Run 4	0.37695	0.039915	0.94011	0.059891	1	1	0.91024	0.95247	0
Run 5	0.38998	0.040393	0.93014	0.069863	0.99737	0.995	0.8978	0.94458	0.005
Mean Values:	0.38873	0.05134	0.93276	0.06724	0.99889	0.998	0.90021	0.94622	0.002

Bigger value of Threshold = .9:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.50529	0.083654	0.86869	0.13131	1	1	0.80329	0.88983	0
Run 2	0.31341	0.057991	0.86522	0.13478	1	1	0.79799	0.88642	0
Run 3	0.36556	0.035207	0.85866	0.14134	1	1	0.78823	0.87963	0
Run 4	0.3871	0.037566	0.85861	0.14139	1	1	0.78811	0.88026	0
Run 5	0.38017	0.042592	0.87358	0.12642	1	1	0.81055	0.89381	0
Mean Values:	0.39031	0.051402	0.86495	0.13505	1	1	0.79763	0.88599	0

Smaller value of Threshold = .1:

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	0.4586	0.073614	0.96339	0.036612	0.97333	0.945	0.97256	0.97268	0.055
Run 2	0.37539	0.065372	0.96508	0.034918	0.97036	0.94	0.97756	0.97388	0.06
Run 3	0.37759	0.038788	0.96508	0.034918	0.9756	0.95	0.97256	0.97388	0.05
Run 4	0.38822	0.040263	0.96333	0.036667	0.97298	0.945	0.9725	0.97246	0.055
Run 5	0.36043	0.039923	0.96511	0.034891	0.9803	0.96	0.96762	0.97353	0.04
Mean Values:	0.39204	0.051592	0.9644	0.035601	0.97451	0.948	0.97256	0.97329	0.052

We found that the optimal value was .1.

With our final values of each parameter this is the final results:
kernel = gaussian
lambda = 1e-20
C = 40
sigmakernel = 5
k = 10
ndim = 70
threshold = .1

From the table below we can see that this yields the highest accuracy of 91% while keeping overfitting and underfitting in check by using mid-range k and threshold values. Precision, specify and false alarm rate all produce excellent, near perfect results.

Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
Label	trainingTime	testingTime	accuracy	errorRate	precision	specificity	sensitivity	fMeasure	falseAlarmRate
Run 1	1.0322	0.11339	0.96008	0.039918	0.96831	0.935	0.97256	0.97014	0.065
Run 2	0.79321	0.079991	0.95505	0.044945	0.96367	0.925	0.97	0.9665	0.075
Run 3	0.79948	0.067095	0.95514	0.044863	0.96099	0.92	0.97262	0.96646	0.08
Run 4	0.72274	0.067496	0.95669	0.043306	0.96678	0.93	0.97	0.96771	0.07
Run 5	0.72727	0.065228	0.96003	0.039973	0.96839	0.935	0.9725	0.97018	0.065
Mean Values:	0.81498	0.07864	0.9574	0.042601	0.96563	0.929	0.97154	0.9682	0.071

Detection Implementation

Sliding Window

In our pedestrian detection system, we combined a sliding window detector with non-maxima suppression to identify pedestrians in images. The sliding window works by defining a fixed window size, which is then moved across the image in steps. At each position, the window crops a region of the image, and features are extracted using our chosen feature extractor, HOG. The cropped features are pre-processed in the same way as during training, including z-score standardization, L2 normalization, and PCA for dimensionality reduction. Once processed, the classifier evaluates the window to determine whether it contains a pedestrian. If a pedestrian is detected, the coordinates and dimensions of the bounding box (x, y, width, height) are recorded.

The window then moves to the next position, with the step size adjusted according to the current scale to ensure adequate coverage without excessive overlap. After scanning the entire image at one scale, the image is resized to a new scale while keeping the window size fixed. This approach allows the classifier to always receive input of the same size as it was trained on, while enabling the detector to identify pedestrians of varying sizes across the image.

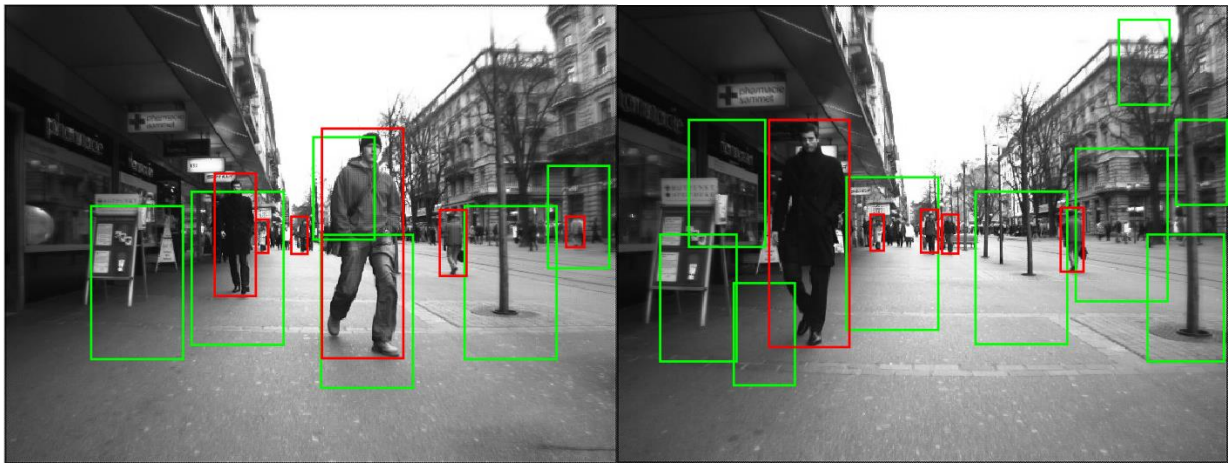
To improve the quality of our detections and reduce redundancy, we applied non-maxima suppression (NMS). Overlapping bounding boxes are a common issue in sliding window detection, as multiple windows may detect the same pedestrian. NMS addresses this by keeping only the strongest detection in each overlapping region and discarding weaker, redundant boxes. This ensures a cleaner set of final detections and improves overall precision.

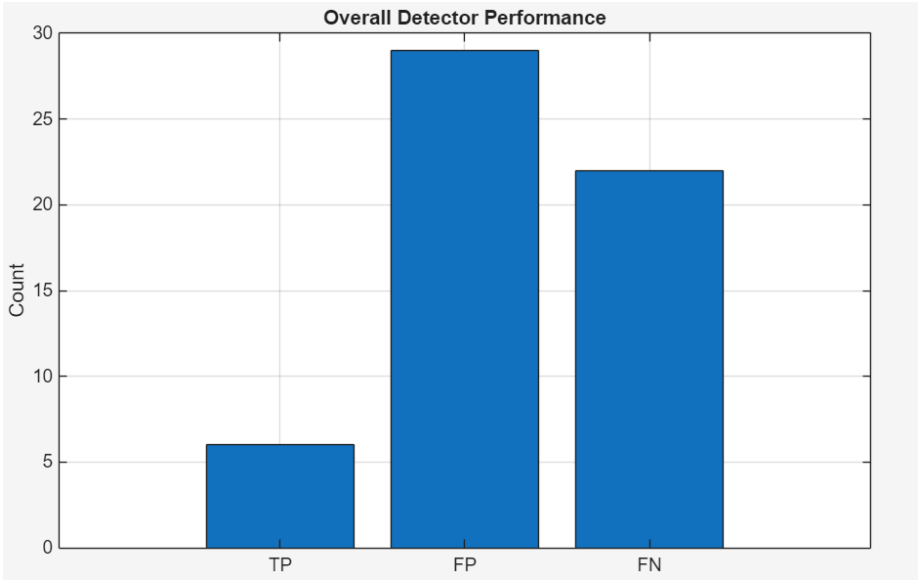
Evaluation

To evaluate performance, we compared the detected bounding boxes against the ground truth provided in the test.dataset file, which specifies the coordinates and dimensions of pedestrians in each frame. Evaluation metrics, including true positives, false positives, and false negatives, were then calculated to assess the effectiveness of the sliding window detector combined with non-maxima suppression. Additionally, we generated an output video showing the bounding boxes overlaid on each frame, providing a visual

demonstration of the system's effectiveness. This video is present in the GitLab, but is just a combination of the testing images, with the bounding boxes overlapped.

Using the SVM model from the optimal values we obtained previously and running on the first 5 images in the dataset, our classifier gave us the following output. The red boxes are the ground truth and the green boxes are the detected pedestrians:

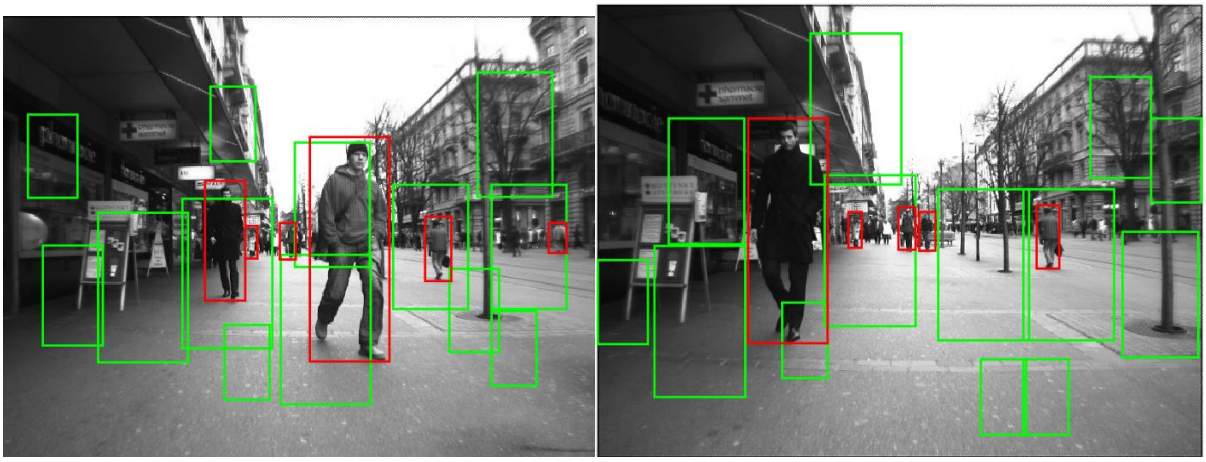


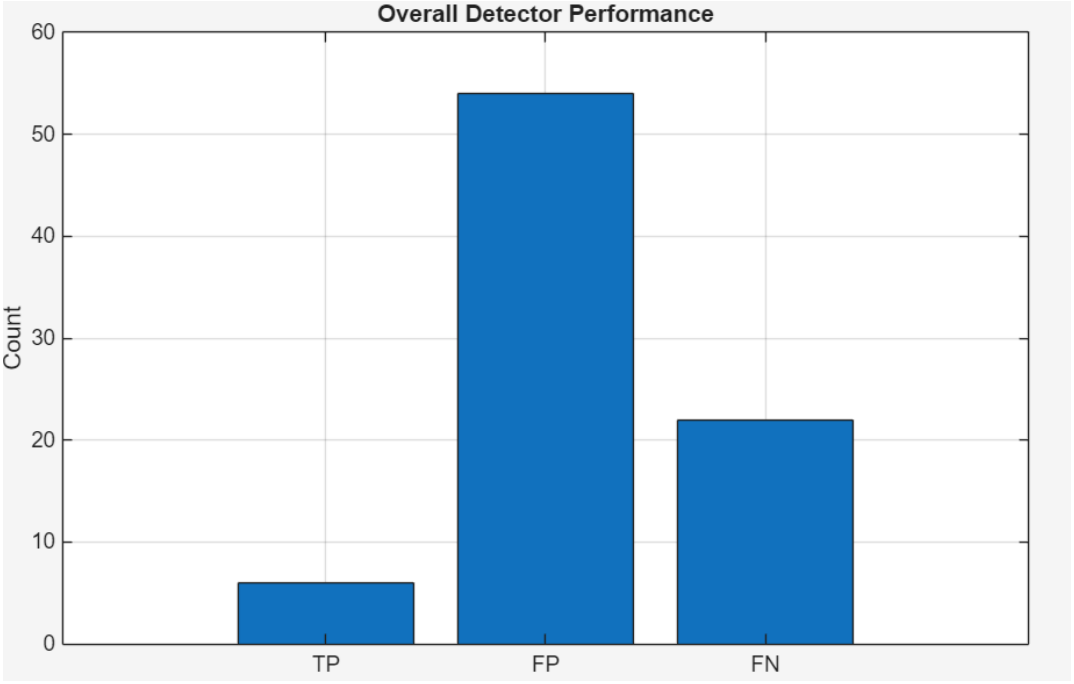


As you can see the detector does not seem to be functioning very well, in the first image it detects a sign and a lamppost as a pedestrian. There are a lot of false positives present, a reason for this being the way the model was trained was on a dataset of a split mixture of pedestrians and non pedestrians equally, whereas in this application the majority of the crops do not contain pedestrians. Another theory is that the model was not trained on enough samples, as the value of k chosen was 10, that means that 1 tenth of the data

available was not used for training, as the model only is only saved as the last fold of the cross validation. Another theory is that there aren't enough diverse scales, as some of the smaller pedestrians don't seem to have a scale of that side.

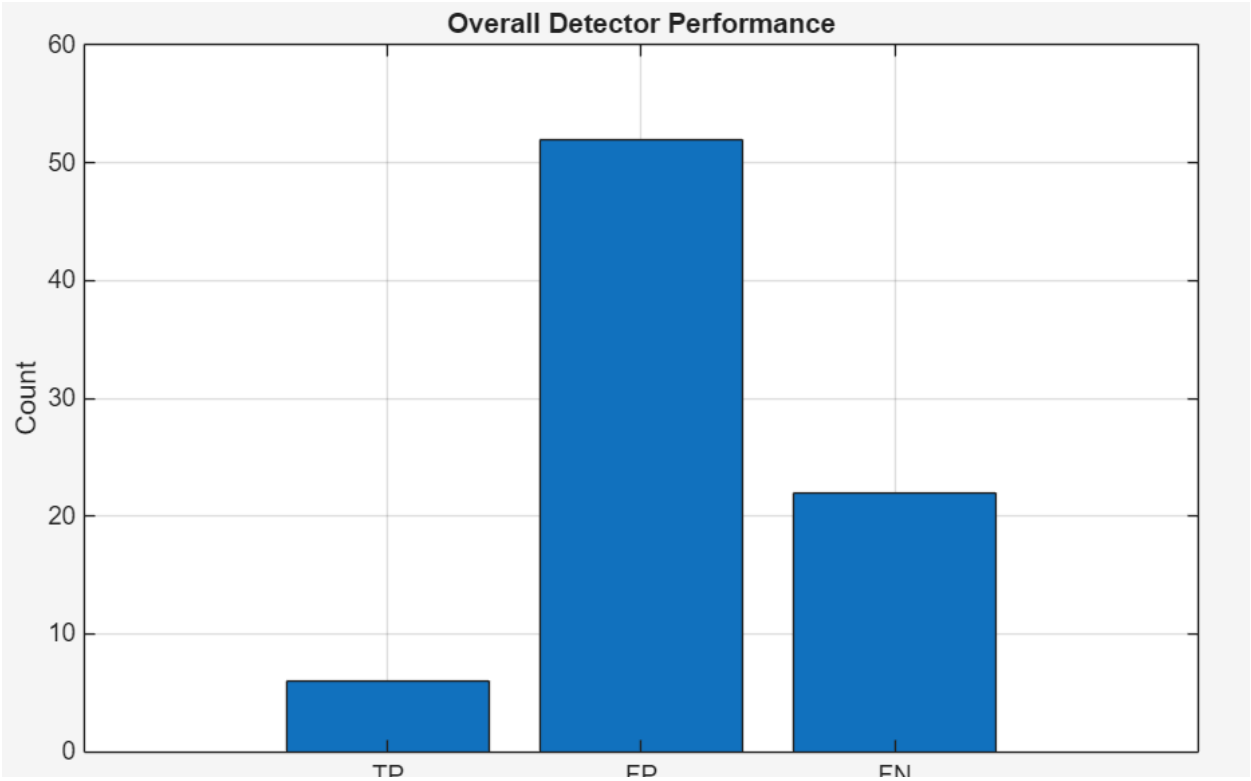
Following these theories the SVM training was changed slightly to train on all the dataset, the value of C was increased to 120 to punish incorrect classifications more and the threshold was increased to .9 to ensure that the model had to be positive for the window to detect a pedestrian and finally, another scale was added to the sliding window. The changes resulted in this output from the sliding window.





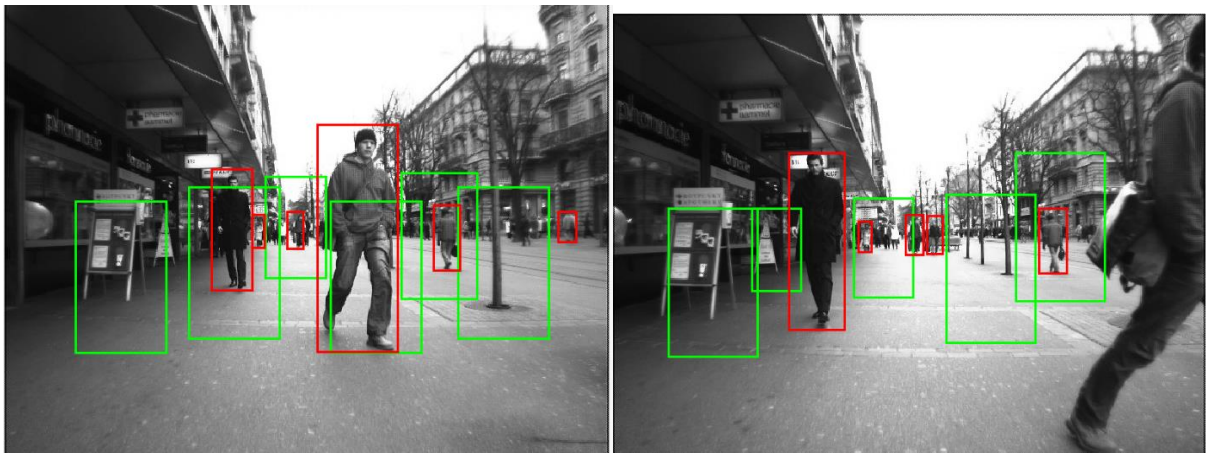
As you can see, this seemed to have the opposite effect, causing much more false positives to be shown instead of reducing them. This led me to the theory that the model could be overfit, learning too much of the training set and not being able to adapt to new

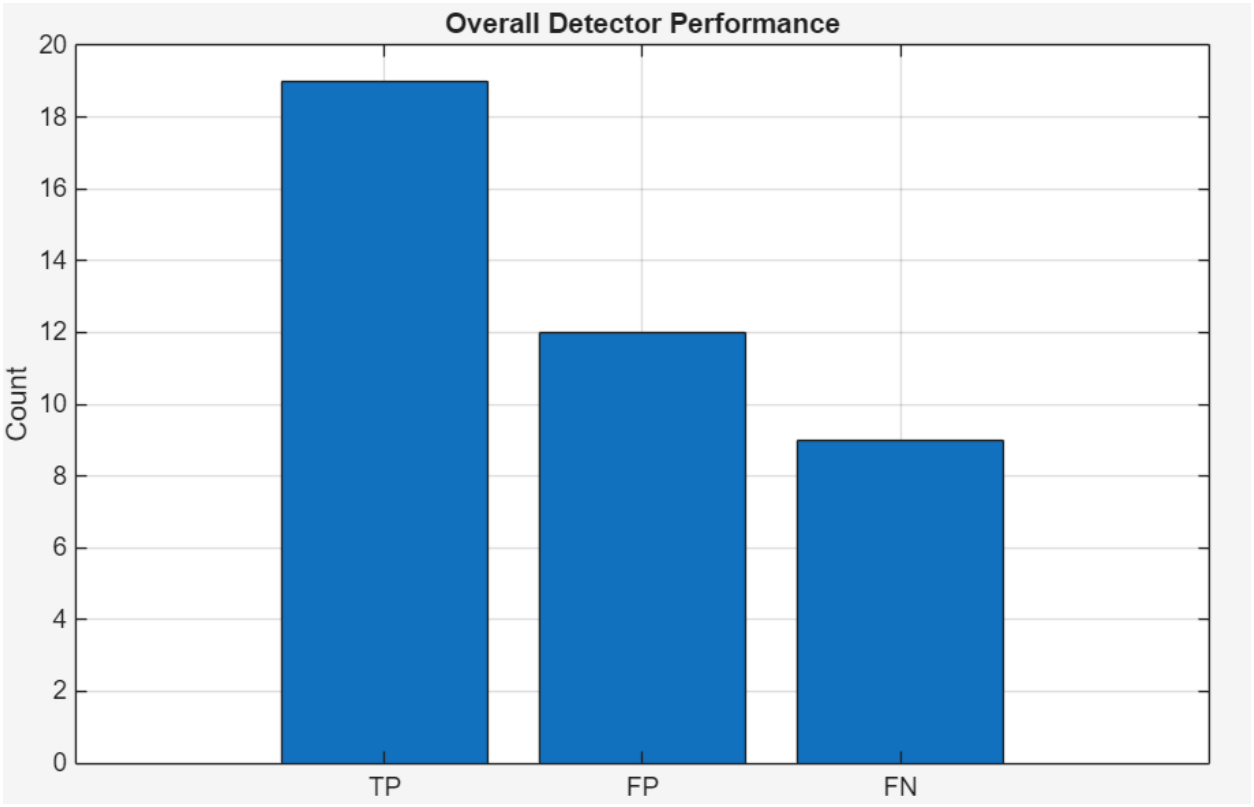
changes. So following this, I went back to using cross validation and reduced the number of folds to 3 so that the model was only being trained on two thirds of the images. This gave us the following output:



This still did not produce the effect I wanted, and I realised that by increasing the value of C, I was not only increasing the cost of false positives, but also false negatives, meaning that my model could be overfit using such a high value of C. Following everything I had learned, I decided that my final version was going to train the model on all of the available data, I was going to use a higher threshold value of .95, and I was going to use the optimum

values we found earlier. This gave us the following output





Conclusion

Throughout this project, we feel that our chosen feature-extraction methods and training strategy yielded a capable model that can reasonably identify pedestrians with accuracy, even under challenging conditions such as variable lighting and image noise. The testing results showed that the model performed consistently across multiple evaluation metrics, including all confusion matrix values. However, certain aspects of the model fall short, with one of the biggest areas of improvement being computational efficiency.

For this project, we prioritized results in the form of confusion matrix values which resulted in us choosing computationally expensive methods, such as SVM, HOG and cross-validation, rather than their faster counterparts i.e. kNN and 50/50 split, resulting in a very slow program. If our application were to be applied to the real world, while it may be able to output accurate results, it would be considered rather slow.

Another area of improvement would be our sliding window. As discussed previously, we did not manage to get it fully functional but with a lot work and parameter tuning we did manage to get a result which showed a lot more TPs than previous runs.

Overall, while the model has improvements that could be made, it is an effective pedestrian detection system which produces consistent, respectable results.